

AgenteRevisa

SISTEMA MULTIAGENTE DE REVISÃO DE CÓDIGO JAVA

Nathalia Barbosa
NFAB

Thales Bezerra
TOB2

Vinicius Barbosa
VRB

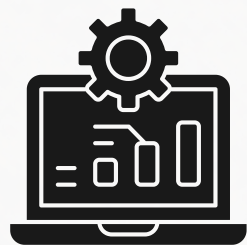
Introdução

- Pipelines de CI/CD frequentemente rodam ferramentas de análise estática e linting quando PRs são abertos, como o Spotbugs, PMD e Checkstyle;
- Quando essas ferramentas indicam erros, os desenvolvedores precisam corrigir parte deles manualmente;
- Visto isso, nossa ferramenta busca automatizar esse processo de refatoração, executando as ferramentas e fazendo refatorações que buscam preservar a lógica inicial do código.
- Além disso, são gerados testes unitários que servem para validar as principais funcionalidades do código

O que o *AgenteRevisa* faz?

- Executa as ferramentas de análise estática Spotbugs, PMD e Checkstyle;
- Resolve os problemas identificados no código;
- Reexecuta iterativamente as ferramentas, para garantir que as alterações não introduziram outros problemas
- Gera testes para a ferramenta, que são executados antes e depois das alterações, e servem para dar uma garantia maior de que as regras de negócio não vão ser quebradas pela modificação

Ferramentas Suportadas



Geração de
Testes



PMD

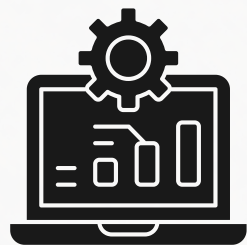


CheckStyle



SpotBugs

Ferramentas Suportadas



Geração de
Testes

- JUnit5
- Os testes auxiliam na preservação da lógica de negócio ao longo da execução dos outros agentes

Ferramentas Suportadas



PMD

- Más prácticas
- Complexidade ciclomática elevada
- Código Morto
- Loops Ineficientes

Ferramentas Suportadas

- Formatação de código
- Identação
- Adesão a regras de estilo



CheckStyle

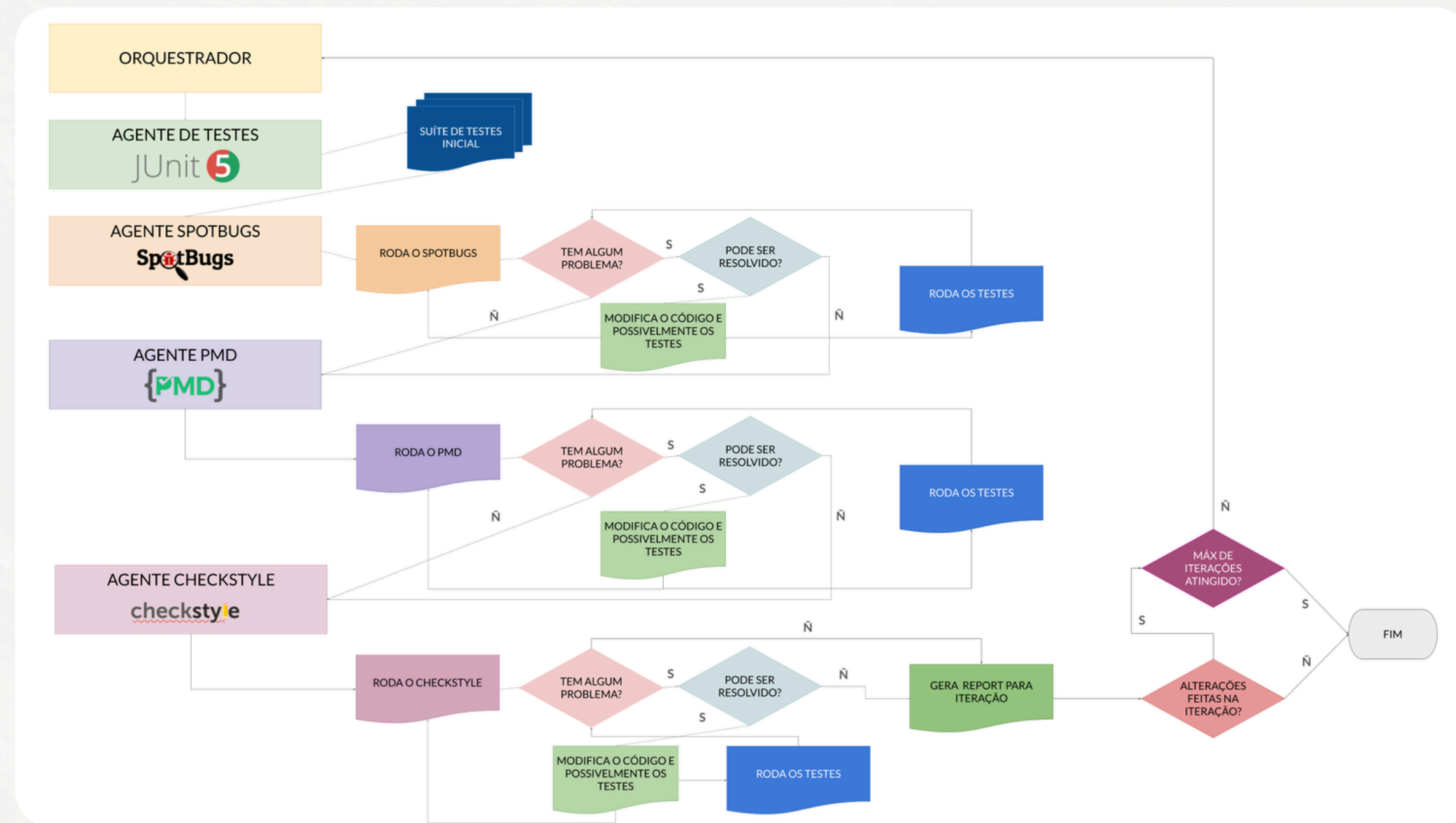
Ferramentas Suportadas

- Segurança
- Thread-safety
- Problemas de compilação



SpotBugs

Fluxo de Execução



Avaliação

- 1 Criação de um dataset com classes que contêm problemas detectados por cada ferramenta
- 2 Execução dos agentes nas classes do dataset
- 3 Análise manual dos outputs

Análise do CheckStyle

- Conseguiu corrigir a maioria dos problemas
- Mas fez muitas mudanças com base apenas no contexto, sem apoio do report da ferramenta (*não é necessariamente ruim*)



CheckStyle

CheckStyle: Exemplo de falha

- Renomeou vários parâmetros sem ser solicitado (inventando nomes).
- A documentação gerada se baseou nesses nomes inventados.

Antes `public void avaliar(int parametro1, String parametro2, double parametro3, boolean parametro4, String parametro5) {
 if (parametro1 >= 18) {`

Depois

```
/**  
 * Avalia se o usuário atende aos requisitos necessários.  
 *  
 * @param idade idade do usuário  
 * @param status status do usuário  
 * @param saldo saldo do usuário  
 * @param ativo indicador de atividade  
 * @param categoria categoria do usuário  
 */  
public void avaliar(int idade, String status, double saldo, boolean ativo, String categoria) {  
    if (idade < 18) {
```

Análise do Spotbugs

- Conseguiu corrigir os principais problemas
- Mas adicionou novos trechos de código em alguns exemplos que quebram retrocompatibilidade



SpotBugs

Spotbugs: Exemplo de falha

- Adicionou do nada uma trava com `if (valorMaximo <= 0) throw new IllegalArgumentException(...)`
- Antes se alguém mandasse 0 ele só devolveria 0. Agora lança exceção em runtime e derruba a aplicação.

Antes

```
public int gerarIdLoteAleatorio(int valorMaximo) {  
    return (int) (random.nextDouble() * valorMaximo);  
}
```

Depois

```
public int gerarIdLoteAleatorio(int valorMaximo) {  
    if (valorMaximo <= 0) {  
        throw new IllegalArgumentException("0 valor maximo deve ser positivo.");  
    }  
    return random.nextInt(valorMaximo);  
}
```

Análise do PMD

- Conseguiu corrigir os principais problemas em poucas iterações
- Causou alguns problemas ocasionais, como geração de código desnecessário e refatoração de partes que não precisavam ser mudadas.



PMD

PMD: Exemplo de falha

- Transformou a interface numa classe e criou um construtor privado.

Antes

```
package com.ficticio.pmd;

public interface StatusDoPedidoConstantes {

    String STATUS_PENDENTE = "PENDENTE";

    String STATUS_PAGO = "PAGO";

    String STATUS_CANCELADO = "CANCELADO";

    String STATUS_ENVIADO = "ENVIADO";

}
```

Depois

```
package com.ficticio.pmd;

public final class StatusDoPedidoConstantes {

    public static final String STATUS_PENDENTE = "PENDENTE";
    public static final String STATUS_PAGO = "PAGO";
    public static final String STATUS_CANCELADO = "CANCELADO";
    public static final String STATUS_ENVIADO = "ENVIADO";

    private StatusDoPedidoConstantes() {
        // Construtor privado para evitar instanciação
    }

}
```